ADVANCED JAVA

with Web Applications | BCAC601

Most Important Exam Questions

16 Frequently Asked 15-Mark Questions — Fully Solved

Subject	Advanced Java with Web Applications
Course Code	BCAC601
Question Type	15-Mark Long Answer Questions (16 Total)
Format	Explanation + Real-world Analogy + Diagram + Java Code + Differences

Table of Contents

Q1. Difference between Java SE and Java EE

Introduction

Java is available in different editions depending on the type of application you want to build. The two most common editions are **Java SE** (Standard Edition) and **Java EE** (Enterprise Edition). Both use the same core Java language, but they are meant for different purposes — SE is for general-purpose desktop/console applications, while EE is for large, multi-user, web-based and enterprise applications.

■ ***Real-world analogy:*** *Think of Java SE as the engine of a car — it lets the car move. Java EE is like adding GPS, AC, music system, and safety airbags on top of that engine to make it ready for a long, comfortable highway journey (i.e., a large enterprise application).*

Java SE (Standard Edition)

- Provides the **core Java** functionality: data types, OOP concepts, Collections, Exception Handling, Multithreading, I/O, and basic JDBC.
- Used to build **desktop applications, console programs, and applets**.
- Does not provide built-in support for web technologies like Servlets or JSP.
- Comes with JDK (Java Development Kit) and JVM.

Java EE (Enterprise Edition)

- Built **on top of** Java SE — it uses everything SE provides and adds extra APIs.
- Provides APIs for **web and enterprise applications**: Servlets, JSP, JSTL, EJB (Enterprise Java Beans), JMS (messaging), JTA (transactions), JAX-RS (REST web services).
- Used to build **large-scale, distributed, multi-tier, web-based applications** such as banking systems, e-commerce portals, and college ERP systems.
- Needs an **application server** like Apache Tomcat, GlassFish, or WildFly to run.

Diagram: Java EE built on top of Java SE

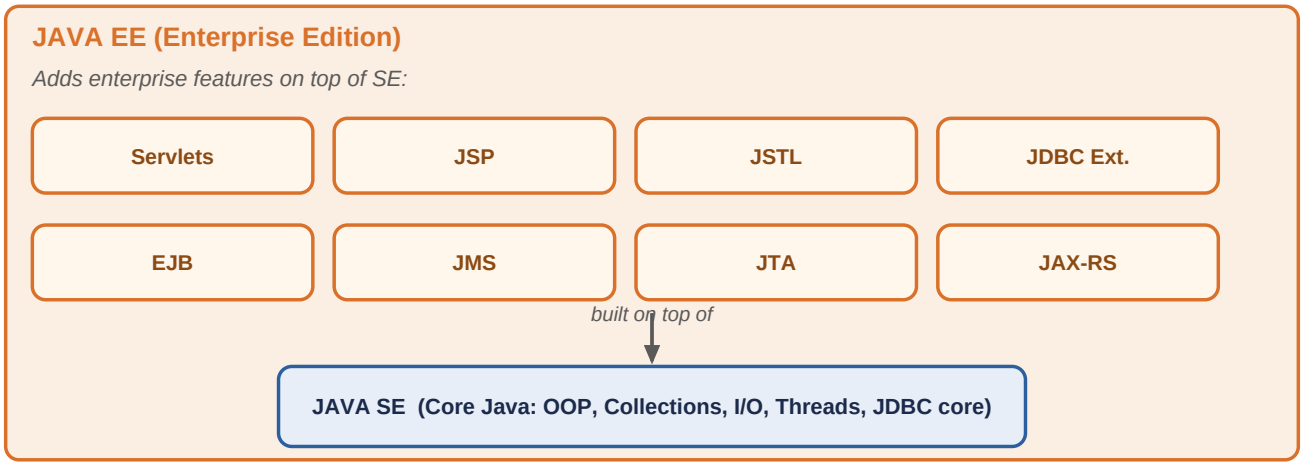


Fig 1.1 — Java EE is not a replacement for Java SE; it is an extra layer of APIs added above it.

Key Differences

Basis	Java SE	Java EE
Full Form	Standard Edition	Enterprise Edition

Basis	Java SE	Java EE
Purpose	Desktop & console applications	Web & enterprise (multi-tier) applications
Core APIs	OOP, Collections, I/O, Threads	Servlets, JSP, EJB, JMS, JTA, JAX-RS
Server Needed	Not required	Requires an application/web server
Scope	Single-user, standalone	Multi-user, distributed, scalable
Built On	Base platform	Built on top of Java SE

Conclusion

In short, Java SE is the **foundation** that every Java program needs, while Java EE is the **extended set of tools** built on that foundation specifically for developing robust, scalable, and secure enterprise-level web applications.

Q2.
JDBC Architecture and JDBC Drivers

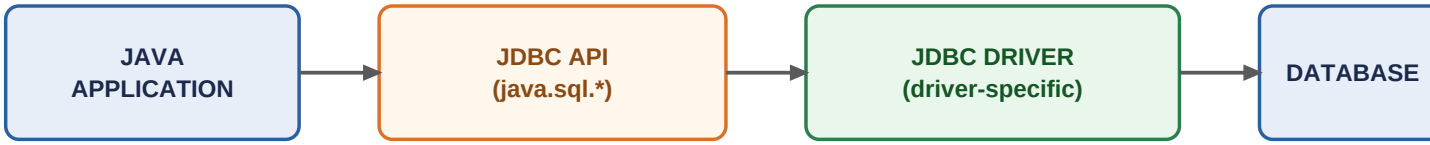
Introduction

JDBC (Java Database Connectivity) is an API that allows a Java program to connect to and interact with a relational database (like MySQL, Oracle) — to send SQL queries and retrieve results, regardless of which database vendor is being used.

■ ***Real-world analogy:*** *JDBC works like a universal travel adapter. Just as one adapter lets your charger work in different countries with different socket types, JDBC lets the same Java code talk to different databases (MySQL, Oracle, etc.) using different drivers underneath.*

JDBC Architecture (2 layers)

- **JDBC API Layer:** Provides the application-to-JDBC-Manager connection. Java code talks to interfaces like `Connection`, `Statement`, `ResultSet` defined in `java.sql` package.
- **JDBC Driver API Layer:** Supports the JDBC Manager-to-Driver connection. The `DriverManager` class loads the correct driver and routes calls from the API layer to the actual database-specific driver.



Java App --> JDBC API --> JDBC Driver --> Database

Fig 2.1 — Flow of a JDBC call from Java application to the database.

Steps to connect using JDBC

Code: Basic JDBC connection flow

```
// Step 1: Load and register the driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Step 2: Establish connection
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/college", "root", "password");

// Step 3: Create Statement
Statement stmt = con.createStatement();

// Step 4: Execute Query
ResultSet rs = stmt.executeQuery("SELECT * FROM students");

// Step 5: Process result
while (rs.next()) {
    System.out.println(rs.getString("name"));
}

// Step 6: Close connection
con.close();
```

Types of JDBC Drivers

Type	Name	Description
Type 1	JDBC-ODBC Bridge Driver	Converts JDBC calls into ODBC calls. Obsolete, slow, platform-dependent.
Type 2	Native-API Driver	Converts JDBC calls into database-specific native client API calls. Needs native library installed.
Type 3	Network Protocol Driver	Sends requests to a middleware server, which translates them for the database. Fully written in Java.
Type 4	Thin Driver (Pure Java)	Directly converts JDBC calls into the database's network protocol. Fastest, platform-independent, most widely used today.

Important: Type 4 (Thin Driver) is the most commonly used and exam-favourite driver because it is 100% Java, requires no extra native software, and gives the best performance.

Conclusion

JDBC architecture cleanly separates the application logic from database-specific details using the API and Driver layers, while JDBC drivers (Types 1-4) determine how that communication actually happens — with Type 4 drivers being the industry standard today.

Q3.
Statement vs PreparedStatement

Introduction

Both `Statement` and `PreparedStatement` are JDBC interfaces used to send SQL queries to a database from a Java program. However, they differ significantly in performance, security, and how they handle SQL queries with changing values.

> ■ ***Real-world analogy:*** *A **Statement** is like writing a fresh handwritten letter every single time you want to send the same message to different people — tiring and error-prone. A **PreparedStatement** is like having a printed template letter with blank spaces (placeholders) where you just fill in the name each time — faster and safer.*

Statement

- Used for executing simple, static SQL queries that do not change.
- The SQL query is **compiled every time** it is executed — even if the same query runs repeatedly with different values, leading to performance overhead.
- Values are directly concatenated into the SQL string, making it **vulnerable to SQL Injection** attacks.

Code: Using Statement

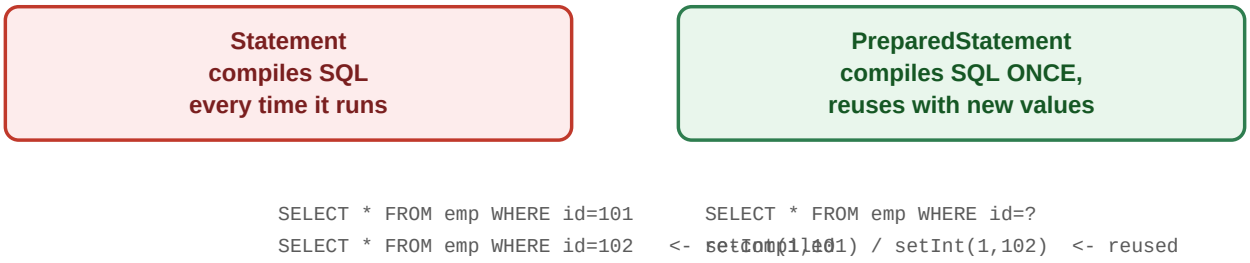
```
Statement stmt = con.createStatement();
String sql = "SELECT * FROM employee WHERE id = " + empId;
ResultSet rs = stmt.executeQuery(sql);
// Risk: if empId comes from user input like "101 OR 1=1",
// it can break the query logic (SQL Injection)
```

PreparedStatement

- Used for executing parameterized (dynamic) SQL queries using **?** placeholders.
- The SQL query is **pre-compiled once** by the database, then reused multiple times with different parameter values — much faster for repeated execution.
- Parameter values are bound safely using setter methods, which **prevents SQL Injection**.

Code: Using PreparedStatement

```
PreparedStatement pstmt = con.prepareStatement(
    "SELECT * FROM employee WHERE id = ?");
pstmt.setInt(1, empId);  // safely binds the value
ResultSet rs = pstmt.executeQuery();
```



PreparedStatement avoids repeated compilation and blocks SQL Injection

Fig 3.1 — Statement recompiles SQL each time; PreparedStatement compiles once and reuses it.

Key Differences

Basis	Statement	PreparedStatement
Query Type	Static SQL	Dynamic / Parameterized SQL
Compilation	Compiled every execution	Compiled once, reused
Performance	Slower for repeated queries	Faster for repeated queries
SQL Injection	Vulnerable	Prevented (parameters are bound, not concatenated)
Placeholders	Not supported	Supported using ?
Interface	java.sql.Statement	java.sql.PreparedStatement (extends Statement)

Conclusion

For one-time, simple queries, Statement is sufficient, but for any query that takes user input or runs repeatedly, **PreparedStatement is always preferred** in real-world applications because it is faster and far more secure.

Q4.
Transaction Management with Commit and Rollback

Introduction

A **transaction** is a group of one or more SQL statements that are executed as a single unit of work — either **all** statements succeed together, or **none** of them are applied. JDBC provides transaction management through the `Connection` interface using `commit()` and `rollback()`.

> ■ ***Real-world analogy:*** *Think of a bank money-transfer: money is deducted from Account A and added to Account B. If the deduction succeeds but the addition fails halfway (say, due to a power cut), the money should not vanish — the whole operation must be undone. This 'all-or-nothing' behaviour is exactly what transaction management guarantees.*

ACID Properties (why transactions matter)

- **Atomicity** — all operations in the transaction complete, or none do.
- **Consistency** — the database moves from one valid state to another.
- **Isolation** — concurrent transactions do not interfere with each other.
- **Durability** — once committed, changes survive even a system crash.

commit() and rollback()

- By default, JDBC runs in **auto-commit mode** — every SQL statement is committed immediately after execution.
- To manage a transaction manually, auto-commit must first be disabled using `con.setAutoCommit(false)`.
- `commit()` permanently saves all changes made since the transaction began.
- `rollback()` undoes all changes made since the last commit, typically called i

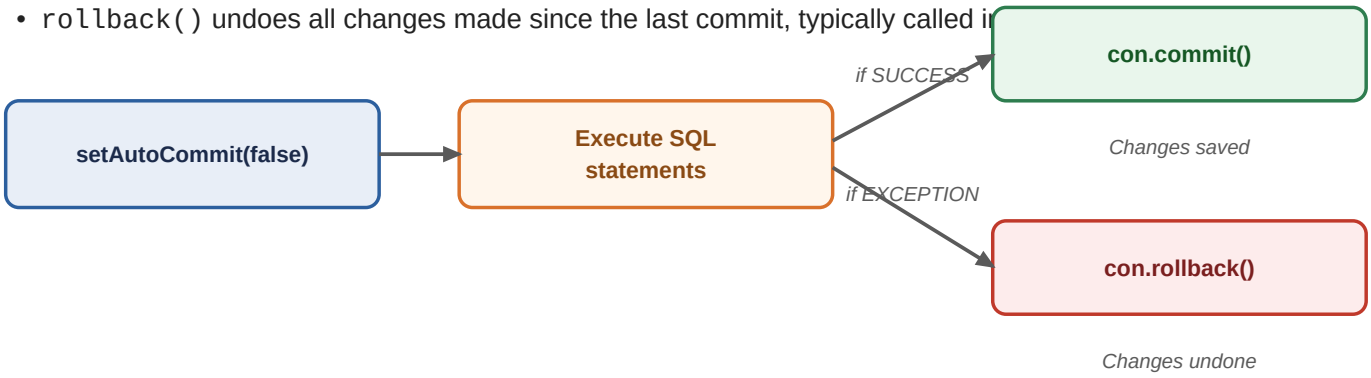


Fig 4.1 — Transaction flow: disable auto-commit, run SQL, commit on success or rollback on failure.

Code Example: Bank Transfer Transaction

Code: Transferring money safely using a transaction

```
try {
    con.setAutoCommit(false);   // start transaction

    PreparedStatement p1 = con.prepareStatement(
        "UPDATE account SET balance = balance - ? WHERE acc_no = ?");
    p1.setDouble(1, 5000); p1.setInt(2, 101);
    p1.executeUpdate();

    PreparedStatement p2 = con.prepareStatement(
        "UPDATE account SET balance = balance + ? WHERE acc_no = ?");
    p2.setDouble(1, 5000); p2.setInt(2, 102);
    p2.executeUpdate();

    con.commit();               // both succeeded -> save permanently
    System.out.println("Transfer successful");

} catch (Exception e) {
    con.rollback();             // undo everything
    System.out.println("Transfer failed, rolled back");
}
```

Conclusion

Transaction management with commit and rollback ensures **data integrity** in multi-step operations. By disabling auto-commit and explicitly controlling when changes are saved or undone, JDBC guarantees that the database never ends up in a half-updated, inconsistent state.

Q5.
JSP Lifecycle

Introduction

JSP (JavaServer Pages) is a technology used to create dynamic web pages by mixing HTML with Java code. Internally, every JSP page is converted into a Servlet before it can run. The **JSP life cycle** describes the complete sequence of phases a JSP page goes through, from the moment it is requested until it is destroyed.

■ ***Real-world analogy:*** *Think of a JSP page like a chef preparing a recipe for the first time: first they translate the handwritten recipe into clear steps (translation), then practice and prepare ingredients once (compilation and init). After that, every time a customer orders the same dish, the chef just cooks and serves quickly (service) — without re-reading the whole recipe again.*

Phases of JSP Life Cycle

- **1. Translation:** The JSP container converts the .jsp file into a Servlet `.java` source file. HTML becomes `out.print()` statements.
- **2. Compilation:** The generated .java file is compiled into a `.class` file by the Java compiler.
- **3. Class Loading & Instantiation:** The container loads the servlet class and creates an object of it.
- **4. Initialization — jspInit():** Called only **once**, right after the object is created. Used to set up resources like DB connections.
- **5. Execution — _jspService():** Called for **every client request**. It processes the request, generates the response, and is auto-generated (cannot be overridden).
- **6. Destruction — jspDestroy():** Called only **once**, when the container decides to remove the JSP from service (e.g., on server shutdown). Used to release resources.

Steps 1 & 2 happen only ONCE (first request) | Step 4 repeats for EVERY request

Fig 5.1 — JSP life cycle: translation and compilation happen once; service repeats per request.

Code Example

Code: hello.jsp demonstrating lifecycle methods

```
<%@ page language="java" %>
<%!
  public void jspInit() {
      System.out.println("JSP Initialized - runs only once");
  }
  public void jspDestroy() {
      System.out.println("JSP Destroyed - runs only once");
  }
%>
<html><body>
  <% System.out.println("Service - runs on every request"); %>
  <h2>Welcome to JSP Life Cycle Demo</h2>
</body></html>
```

Important: jspInit() and jspDestroy() run only ONCE in the whole lifecycle, but _jspService() runs EVERY time a new request comes in — this is the most commonly asked exam point.

Conclusion

The JSP life cycle ensures that the heavy work (translation and compilation) is done only once, while the lightweight servicing of requests happens repeatedly and efficiently — making JSP pages fast after the very first request.

Q6.
JSP Implicit Objects

Introduction

Implicit objects are objects that are **automatically created by the JSP container** for every JSP page — the programmer does not need to declare or instantiate them manually. They can be used directly inside scriptlets and expressions to interact with requests, responses, sessions, and the application.

■ ***Real-world analogy:*** *Implicit objects are like the standard furniture that comes pre-installed in a rented flat — you don't have to buy or set up the bed, fan, or lights yourself; they are already there for you to use the moment you move in.*

The 9 Implicit Objects in JSP

Implicit Object	Type	Purpose
request	HttpServletRequest	Holds client request data (form parameters, headers).
response	HttpServletResponse	Used to send response back to the client.
out	JspWriter	Used to write output/content to the response page.
session	HttpSession	Maintains user session data across multiple requests.
application	ServletContext	Shared data/resources accessible by the entire web app.
config	ServletConfig	Configuration information for the current JSP/Servlet.
page	Object (this)	Refers to the current JSP page instance itself.
pageContext	PageContext	Provides access to all other scopes (page, request, session, application).
exception	Throwable	Used in error pages to access the exception that occurred.

Code Example

Code: Using request, session, out and application together

```
<%
    String name = request.getParameter("uname");
    session.setAttribute("user", name);
    out.println("Hello, " + name + "! Welcome.");
    application.setAttribute("visits",
        (int) application.getAttribute("visits") + 1);
%>
```

Why are they useful?

- They save the programmer from writing repetitive boilerplate code for common tasks.
- They give direct access to request data, session data, and application-wide data without extra setup.
- `pageContext` is special — it can access data from **all four scopes** (page, request, session, application) using a single object.

Conclusion

JSP implicit objects greatly simplify development by providing ready-made access to the request/response cycle, session tracking, and application-wide resources, making JSP code shorter and easier to write compared to plain Servlets.

Q7.
JSTL Core Tags with Examples

Introduction

JSTL (JSP Standard Tag Library) is a collection of pre-built tags that allow developers to implement common logic — like loops, conditionals, and iteration — **without writing Java scriptlet code** inside JSP pages. The **Core tag library** (prefix c) is the most frequently used part of JSTL, handling variable support, flow control, and URL management.

■ ***Real-world analogy:*** *Writing Java scriptlets inside JSP is like mixing cooking oil into your tea — messy and hard to maintain. JSTL tags let you write clean HTML-like tags (e.g. <c:if>, <c:forEach>) instead, keeping presentation and logic neatly separated.*

To use JSTL, declare the taglib directive

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

Important JSTL Core Tags

Tag	Purpose
	Displays the value of an expression (like print/echo).
	Sets/stores a value into a variable in a given scope.
	Performs simple conditional (if-only) logic.
, ,	Performs if-else-if / switch-like conditional logic.
	Loops over a collection, array, or a fixed number of times.
	Iterates over tokens of a string split by a delimiter.
	Builds a URL, useful for adding parameters dynamically.
	Includes content from another resource/URL into the page.

Code Example: c:if and c:forEach

Code: Conditional check and loop using JSTL core tags

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<c:set var="marks" value="75" />

<c:if test="${marks >= 40}">
    <p>Result: PASS</p>
</c:if>

<ul>
<c:forEach var="i" begin="1" end="5">
    <li>Item number: <c:out value="${i}" /></li>
</c:forEach>
</ul>
```

Advantages of JSTL

- Removes the need for Java scriptlets (<% %>), making JSP pages cleaner and easier to read.
- Encourages separation of business logic from presentation logic (closer to MVC principles).
- Reduces development and debugging time since tags are pre-tested and reusable.

Conclusion

JSTL Core tags provide a tag-based, scriptless way to add logic to JSP pages. By replacing Java code with simple XML-like tags such as <c:if> and <c:forEach>, they make JSP development cleaner, more maintainable, and closer to good MVC design practice.

Q8.
Servlet Lifecycle

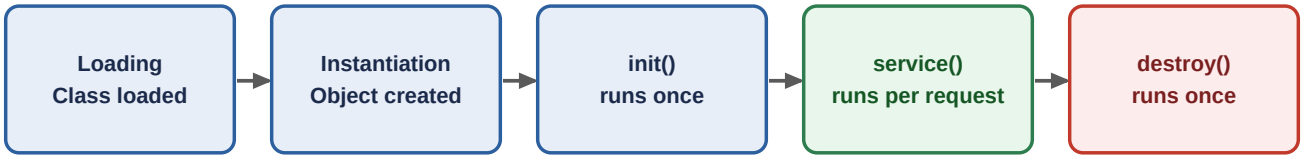
Introduction

A Servlet is a Java class that handles client requests and generates dynamic responses on a web server. The **Servlet life cycle** defines the complete set of phases a servlet goes through — from being loaded into memory to being destroyed — managed entirely by the **Servlet Container** (e.g., Apache Tomcat).

■ ***Real-world analogy:*** *A servlet's life is like a single employee hired for a shop: they are hired once (loading/instantiation), trained once on shop rules (init), then they serve many customers one after another all day (service), and finally they retire when the shop closes (destroy).*

Phases of Servlet Life Cycle

- **1. Loading the Servlet class:** The container loads the servlet's .class file into memory, usually on the first request or at server startup.
- **2. Instantiation:** The container creates a single object/instance of the servlet using its default constructor.
- **3. Initialization — init():** Called **only once**, immediately after instantiation. Used for one-time setup like loading configuration or opening a DB pool.
- **4. Request Handling — service():** Called **for every client request**. It determines the HTTP method (GET/POST) and calls doGet()/doPost() accordingly.
- **5. Destruction — destroy():** Called **only once**, when the container shuts down the servlet (e.g., during server shutdown), used to release resources.



service() repeats for EVERY client request - this is the most active phase

Fig 8.1 — Servlet life cycle: init() and destroy() each run once; service() runs per request.

Code Example

Code: HelloServlet.java showing all 3 overridable lifecycle methods

```java
public class HelloServlet extends HttpServlet {

    public void init() {
        System.out.println("Servlet Initialized - runs once");
    }

    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h2>Hello from Servlet!</h2>");
    }

    public void destroy() {
        System.out.println("Servlet Destroyed - runs once");
    }
}
```

> **Important:** Only ONE instance of a servlet is normally created by the container, no matter how many users send requests — multiple threads share that single instance to handle concurrent requests, which is why servlets must be written thread-safe.

Conclusion

The Servlet life cycle, fully managed by the container, ensures that costly setup (init) and cleanup (destroy) are done only once, while the actual request processing (service) is repeated efficiently for every client — making servlets fast and scalable.

Q9.
ServletContext vs ServletConfig

Introduction

Both `ServletContext` and `ServletConfig` are interfaces used to pass configuration information to servlets, but they differ in **scope** — one is shared application-wide, while the other is specific to a single servlet.

> ■ ***Real-world analogy:*** *ServletContext is like the Wi-Fi router of an entire office building — every employee (servlet) in every room can connect to it and share the same settings. ServletConfig is like a personal nameplate on one employee's own desk — it belongs only to that one employee and nobody else can use it.*

ServletConfig

- Represents configuration information for a **single specific servlet**.
- Parameters are defined inside the <servlet> tag in `web.xml` using <init-param>.
- Retrieved using `getServletConfig().getInitParameter("name")`.
- Used for settings that are relevant only to that one servlet.

ServletContext

- Represents the **entire web application** — there is only **one ServletContext per application**, shared by all servlets/JSPs in it.
- Parameters are defined using <context-param> at the application level in `web.xml`.
- Retrieved using `getServletContext().getInitParameter("name")`.
- Also used to share data between servlets using `setAttribute()`/`getAttribute()`.

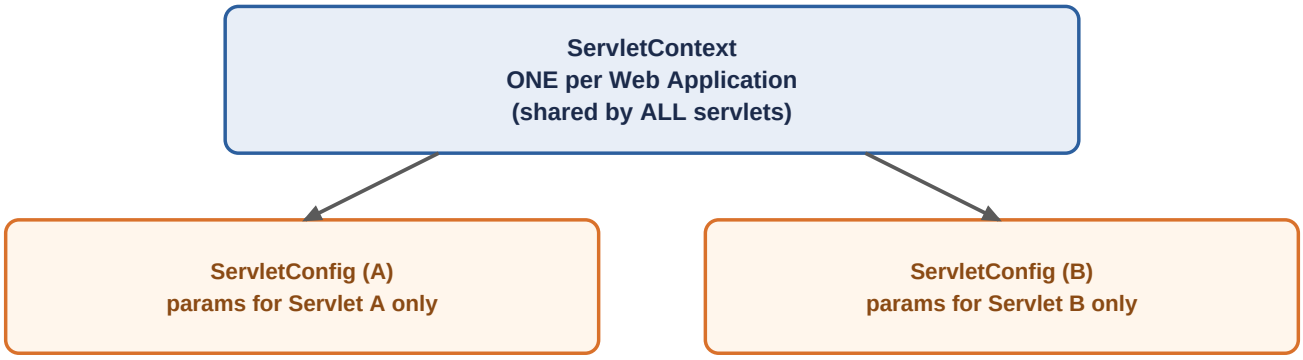


Fig 9.1 — One ServletContext is shared app-wide; each servlet gets its own private ServletConfig.

Code Example

Code: Defining and reading context-param vs init-param

```
// web.xml
<context-param>
    <param-name>siteName</param-name>
    <param-value>MyCollegePortal</param-value>
</context-param>

<servlet>
    <servlet-name>LoginServlet</servlet-name>
    <init-param>
        <param-name>maxAttempts</param-name>
        <param-value>3</param-value>
    </init-param>
</servlet>

// Inside LoginServlet.java
String site = getServletContext().getInitParameter("siteName"); // app-wide
String attempts = getServletConfig().getInitParameter("maxAttempts"); // this servlet only
```

Key Differences

Basis	ServletConfig	ServletContext
Scope	Specific to one servlet	Shared across entire web application
Number of objects	One per servlet	Only one per application
Defined using	inside	at app level
Used for	Servlet-specific settings	Application-wide settings / inter-servlet data sharing

Conclusion

In short, ServletConfig is private configuration for one servlet, whereas ServletContext is the shared, application-wide configuration and communication channel accessible to every component in the web application.

Q10.
Session Management using Cookies and HttpSession

Introduction

HTTP is a **stateless protocol** — each request is treated independently, and the server does not remember anything about a previous request from the same user. **Session management** is the technique used to overcome this limitation, allowing the server to recognize and track a particular user across multiple requests.

> ■ ***Real-world analogy:*** *Imagine visiting a large mall (the server) and forgetting everything after each step you take (statelessness) — annoying! A wristband given at entry (Cookie/Session ID) lets every shop and gate in the mall instantly recognize who you are without asking your details again.*

1. Session Management using Cookies

- A **Cookie** is a small piece of data sent by the server and stored on the client's browser.
- On every later request, the browser automatically sends this cookie back to the server, allowing identification of the user.
- Cookies can be persistent (stored with an expiry time) or non-persistent (deleted when browser closes).

Code: Creating and reading a Cookie

```
Cookie c = new Cookie("username", "indra");
c.setMaxAge(60*60);          // valid for 1 hour
response.addCookie(c);       // sent to browser

// Reading cookie on a later request
Cookie[] cookies = request.getCookies();
for (Cookie ck : cookies) {
    if (ck.getName().equals("username"))
        System.out.println(ck.getValue());
}
```

2. Session Management using HttpSession

- `HttpSession` is a server-side object that stores user-specific data for the duration of that user's visit.
- The server identifies which session belongs to which user using a unique `JSESSIONID`, usually passed via a cookie automatically.
- More secure than cookies alone, since the actual data stays on the server, not the browser.

Code: Creating and reading data via HttpSession

```
HttpSession session = request.getSession();
session.setAttribute("user", "indra");
session.setMaxInactiveInterval(30*60);  // 30 min timeout

// On a later request
String user = (String) session.getAttribute("user");
```

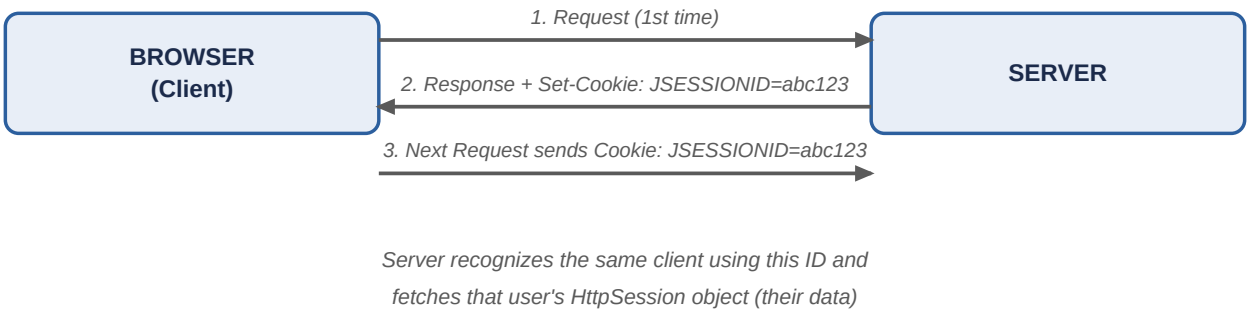


Fig 10.1 — JSESSIONID cookie lets the stateless server recognize the same user on every request.

Cookies vs HttpSession — Quick Comparison

Basis	Cookie	HttpSession
Storage location	Client (browser)	Server
Security	Less secure (visible/editable on client)	More secure (data hidden on server)
Data size limit	Small (~4KB)	Can store larger, complex objects
Lifetime control	Set manually via setMaxAge()	Controlled via setMaxInactiveInterval()

Conclusion

Both cookies and HttpSession solve the same core problem — making a stateless HTTP protocol feel stateful — but HttpSession is generally preferred for sensitive or large data since it keeps information securely on the server while only a small session ID travels through the cookie.

Q11.
MVC Architecture

Introduction

MVC (Model-View-Controller) is a software design pattern that divides a web application into **three interconnected components**, each handling a separate responsibility. In Java web applications, this is typically implemented using JavaBeans/DAO classes as the Model, Servlets as the Controller, and JSP pages as the View.

■ ***Real-world analogy:*** *Think of a restaurant: the **kitchen** (Model) prepares and stores the food/data, the **waiter** (Controller) takes your order, passes it to the kitchen, and brings the food back, and the **plate presentation on your table** (View) is what you actually see and interact with. Each one does its own job without interfering with the others.*

The Three Components

- **Model:** Represents the application's data and business logic. Usually built using JavaBeans or DAO classes that interact with the database. It has no knowledge of how the data will be displayed.
- **View:** Responsible only for presenting data to the user — typically a JSP page that renders HTML output. It contains no business logic.
- **Controller:** A Servlet that receives client requests, calls the appropriate Model logic to process data, and then forwards the result to the correct View for display.

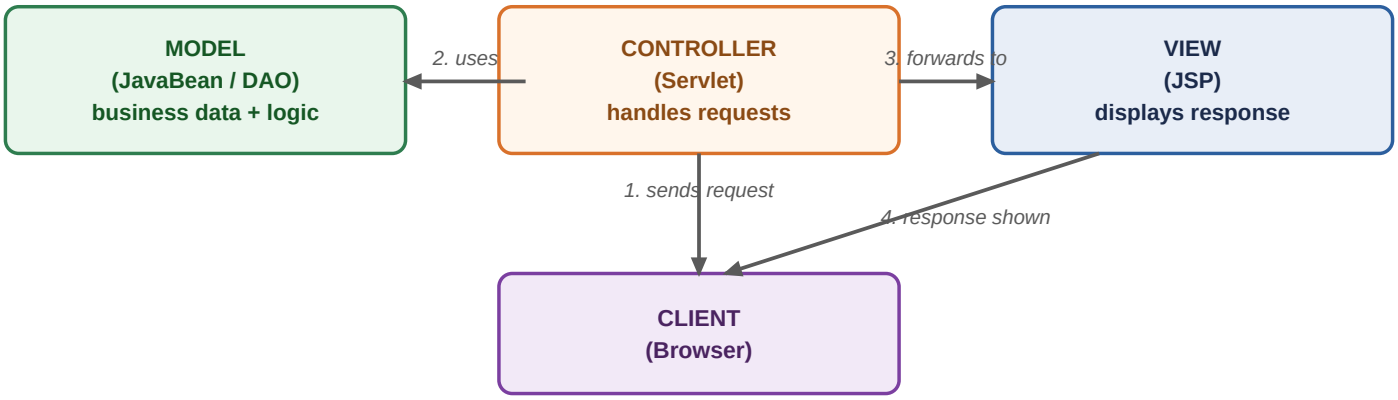


Fig 11.1 — Request flow in MVC: Client to Controller to Model to View to Client.

Code Example

Code: Simple Model-Controller-View flow

```
// Model: Student.java (JavaBean)
public class Student {
    private String name; private int marks;
    public void setName(String n){ name = n; }
    public String getName(){ return name; }
    // similar getters/setters for marks
}

// Controller: StudentServlet.java
protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    Student s = new Student();
    s.setName("Indra");
    req.setAttribute("student", s);
    req.getRequestDispatcher("view.jsp").forward(req, res);  // -> View
}

// View: view.jsp
<p>Name: <%= ((Student)request.getAttribute("student")).getName() %></p>
```

Advantages of MVC

- **Separation of concerns:** business logic, presentation, and request handling are kept independent, making the application easier to manage.
- **Easier maintenance:** changing the look of a page (View) does not affect business logic (Model), and vice versa.
- **Reusability:** the same Model can be reused by multiple Views or Controllers.
- **Parallel development:** different developers can work on Model, View, and Controller simultaneously.

Conclusion

MVC architecture brings structure and discipline to web application development by cleanly separating data (Model), presentation (View), and request-handling logic (Controller) — resulting in code that is easier to test, maintain, and scale.

Q12.
Filters in Servlet

Introduction

A **Filter** is an object that can intercept and process requests **before** they reach a servlet, and responses **after** the servlet has processed them — without the servlet itself needing to know about it. Filters are defined by implementing the `javax.servlet.Filter` interface.

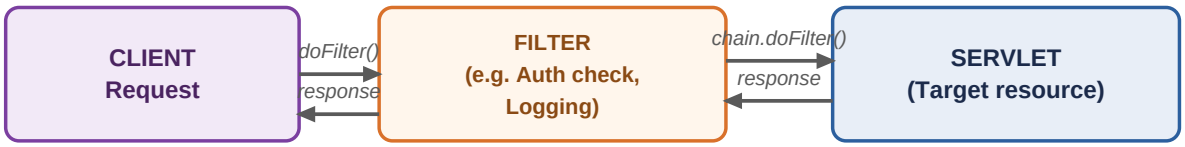
> ■ ***Real-world analogy:*** *A filter is like a security guard and metal detector at the entrance of an exam hall. Every student (request) must pass through the guard before reaching their seat (servlet) — the guard can check ID, log entry time, or even stop unauthorized students from entering at all, without the exam itself needing to know how that checking is done.*

Common Uses of Filters

- **Authentication and Authorization:** checking if a user is logged in before allowing access to a servlet.
- **Logging and Auditing:** recording details of every incoming request.
- **Data Compression:** compressing response data before sending it to the client.
- **Input Validation:** sanitizing or validating request parameters.
- **Encryption/Decryption** of sensitive request or response data.

Filter Lifecycle Methods

- `init(FilterConfig config)` — called once when the filter is first created.
- `doFilter(request, response, chain)` — called for every request; contains the actual filtering logic. Must call `chain.doFilter()` to pass control to the next filter/servlet.
- `destroy()` — called once when the filter is taken out of service.



Filter can inspect/modify request BEFORE it reaches the servlet, and response AFTER

Fig 12.1 — A Filter sits between the client and the servlet, processing both request and response.

Code Example

Code: A simple authentication Filter

```
public class AuthFilter implements Filter {

    public void doFilter(ServletRequest req, ServletResponse res,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest hreq = (HttpServletRequest) req;
        if (hreq.getSession().getAttribute("user") == null) {
            ((HttpServletResponse) res).sendRedirect("login.jsp");
            return;   // stop here - do NOT continue the chain
        }
        chain.doFilter(req, res);  // continue to the actual servlet
    }
}

// web.xml mapping
<filter>
    <filter-name>AuthFilter</filter-name>
    <filter-class>com.app.AuthFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>AuthFilter</filter-name>
    <url-pattern>/dashboard</url-pattern>
</filter-mapping>
```

Important: If `chain.doFilter()` is not called, the request will never reach the target servlet — this is exactly how filters can block unauthorized access.

Conclusion

Filters provide a clean, reusable way to perform cross-cutting tasks like authentication, logging, and validation on multiple servlets, without duplicating that logic inside every individual servlet — making applications more modular and maintainable.

Q13.
Hibernate Architecture

Introduction

Hibernate is a Java **ORM (Object-Relational Mapping)** framework that maps Java classes to database tables, allowing developers to work with Java objects instead of writing raw SQL queries. Its architecture is organized into several layers that together manage the connection between the Java application and the underlying database.

■ ***Real-world analogy:*** *Hibernate works like a translator at an international conference. The Java application speaks 'Java objects', the database understands only 'SQL tables' — Hibernate sits in between, quietly translating every object operation into the right SQL statement and back, so neither side has to learn the other's language directly.*

Core Components of Hibernate Architecture

- **Configuration:** Reads settings from `hibernate.cfg.xml` (DB driver, URL, username, password, dialect) and mapping files/annotations.
- **SessionFactory:** A heavyweight, thread-safe object created **once per application** from the Configuration. It is used to create `Session` objects.
- **Session:** A lightweight, single-threaded object representing a conversation between the application and the database. It is used to save, update, delete, and fetch objects.
- **Transaction:** Manages units of work, ensuring operations either fully complete or are rolled back.
- **Query / Criteria:** Used to retrieve objects from the database using HQL or criteria-based API instead of raw SQL.

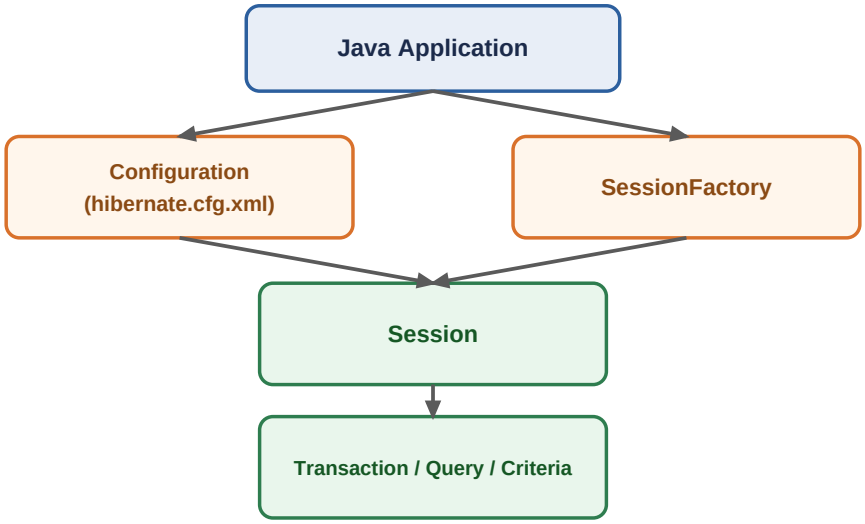


Fig 13.1 — Hibernate architecture: Configuration builds SessionFactory; SessionFactory creates Sessions for actual database operations.

Code Example: Setting up Hibernate

Code: Building SessionFactory and opening a Session

```
// hibernate.cfg.xml (simplified)
<hibernate-configuration><session-factory>
  <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver</property>
  <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/college</property>
  <property name="hibernate.dialect">
      org.hibernate.dialect.MySQLDialect</property>
  <mapping class="com.app.Student"/>
</session-factory></hibernate-configuration>

// Java code to start a session
Configuration cfg = new Configuration().configure();
SessionFactory factory = cfg.buildSessionFactory();
Session session = factory.openSession();
```

Conclusion

Hibernate's layered architecture — Configuration, SessionFactory, Session, and Transaction/Query — cleanly separates one-time setup from the actual day-to-day database operations, making persistence logic simpler and far less repetitive than plain JDBC.

Q14.
Advantages of Hibernate over JDBC

Introduction

JDBC requires developers to manually write SQL queries, manage connections, and convert data between Java objects and database tables. Hibernate, being an ORM framework, automates most of this work, offering several advantages that make development faster, safer, and more maintainable.

■ ***Real-world analogy:*** *Using plain JDBC is like manually packing and unpacking every grocery item one by one every single time you shop. Hibernate is like having a smart trolley that automatically sorts, labels, and stores everything in the right place in your kitchen — saving repeated manual effort each time.*

Key Advantages of Hibernate

- **No boilerplate SQL:** Hibernate automatically generates SQL queries internally, so developers do not need to write repetitive INSERT/UPDATE/DELETE/SELECT statements.
- **Database independence:** Switching databases (e.g., MySQL to Oracle) only requires changing the `dialect` in configuration — Hibernate generates DB-specific SQL automatically.
- **Automatic mapping:** Java objects are automatically mapped to table rows using annotations or XML, removing manual ResultSet-to-object conversion.
- **Caching support:** Built-in first-level (Session) and optional second-level cache reduce repeated database hits, improving performance.
- **HQL (Hibernate Query Language):** An object-oriented query language that works with Java class names/properties instead of raw table/column names.
- **Automatic transaction and connection management:** reduces manual connection-handling code and the risk of connection leaks.
- **Lazy loading:** related data is fetched only when actually needed, improving performance for large object graphs.

Quick Comparison

Basis	JDBC	Hibernate
SQL Queries	Written manually by the developer	Generated automatically by Hibernate
Database portability	SQL often DB-specific, hard to switch	Easy switch via dialect change
Object mapping	Manual ResultSet -> object conversion	Automatic ORM mapping
Caching	Not built-in	Built-in first & second level cache
Query Language	Plain SQL	HQL (object-oriented) + native SQL support
Code volume	More boilerplate code	Significantly less code

Important: Hibernate does not replace JDBC — it actually uses JDBC internally to talk to the database. It simply adds an abstraction layer on top to remove repetitive manual work.

Conclusion

While JDBC gives full manual control, Hibernate provides a higher level of abstraction that reduces development time, improves database portability, and adds useful features like caching and lazy loading —

making it the preferred choice for large enterprise applications.

Q15.
CRUD Operations in Hibernate

Introduction

CRUD stands for **Create, Read, Update, and Delete** — the four basic operations performed on persistent data. Hibernate provides simple, ready-made `Session` methods to perform each of these operations directly on Java objects, without writing any SQL.

> ■ ***Real-world analogy:*** *Think of Hibernate's Session as a librarian managing a library catalogue. To **add** a new book you hand it to the librarian (save), to **find** a book you ask them by its ID (get), to **update** details you hand back a corrected book (update), and to **remove** a book you ask them to discard it (delete) — you never touch the raw catalogue file (SQL/database) yourself.*

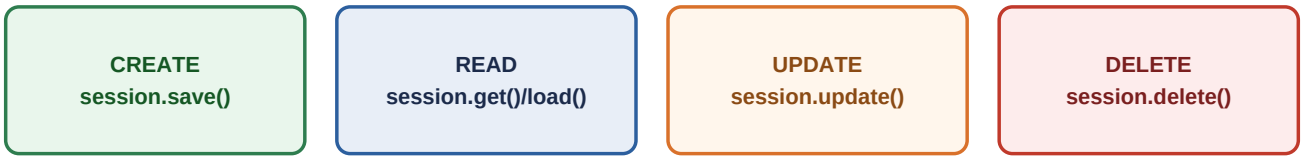


Fig 15.1 — The four CRUD operations and their corresponding Hibernate Session methods.

Entity Class Used in Examples

Code: Student.java entity mapped to the 'student' table

```
@Entity
@Table(name = "student")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private int marks;
    // getters and setters
}
```

1. CREATE — Saving a new record

Code: Create

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Student s = new Student();
s.setName("Indra"); s.setMarks(85);
session.save(s);     // INSERT INTO student ...

tx.commit();
session.close();
```

2. READ — Fetching a record

Code: Read

```
Session session = factory.openSession();
Student s = session.get(Student.class, 1);  // SELECT ... WHERE id=1
System.out.println(s.getName());
session.close();
```

3. UPDATE — Modifying an existing record

Code: Update

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Student s = session.get(Student.class, 1);
s.setMarks(90);          // change value
session.update(s);       // UPDATE student SET marks=90 WHERE id=1

tx.commit();
session.close();
```

4. DELETE — Removing a record

Code: Delete

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Student s = session.get(Student.class, 1);
session.delete(s);       // DELETE FROM student WHERE id=1

tx.commit();
session.close();
```

> **Important:** save(), update(), and delete() must be wrapped in a Transaction and followed by commit() — otherwise the changes will not be permanently reflected in the database.

Conclusion

Hibernate's CRUD methods — save(), get(), update(), and delete() — let developers manage persistent data purely through Java objects, completely hiding the underlying SQL and making database operations significantly simpler than plain JDBC.

Q16.
HQL with Examples

Introduction

HQL (Hibernate Query Language) is an **object-oriented query language** similar to SQL, but instead of operating on table and column names, it operates on **entity class names and their properties**. Hibernate internally translates HQL into the appropriate SQL for the configured database.

■ ***Real-world analogy:*** *Plain SQL asks the database directly: 'give me rows from the student table where the marks column is greater than 80'. HQL instead asks Hibernate in Java terms: 'give me Student objects whose marks property is greater than 80' — and Hibernate quietly converts this into the correct SQL for whichever database is in use.*

Why HQL instead of plain SQL?

- **Database independent:** the same HQL query works regardless of the underlying database, since Hibernate converts it using the configured dialect.
- **Works with objects:** results are returned as actual Java objects, not raw rows, so no manual mapping is needed.
- **Supports OOP features:** HQL understands inheritance, polymorphism, and associations between entities, which plain SQL cannot.

Basic HQL Syntax

```
FROM ClassName [WHERE condition] [ORDER BY property]
```

Code Examples

Code: Common HQL queries — SELECT, WHERE, projection, UPDATE, DELETE

```
Session session = factory.openSession();

// 1. Select all students
Query q1 = session.createQuery("FROM Student");
List<Student> list1 = q1.list();

// 2. Select students with marks greater than 80
Query q2 = session.createQuery("FROM Student s WHERE s.marks > 80");
List<Student> list2 = q2.list();

// 3. Select only specific properties (projection)
Query q3 = session.createQuery("SELECT s.name FROM Student s");
List<String> names = q3.list();

// 4. Update using HQL
Query q4 = session.createQuery(
    "UPDATE Student SET marks = 95 WHERE id = 1");
q4.executeUpdate();

// 5. Delete using HQL
Query q5 = session.createQuery("DELETE FROM Student WHERE id = 2");
q5.executeUpdate();

session.close();
```

HQL vs SQL — Quick Comparison

Basis	SQL	HQL
Operates on	Tables and columns	Entity classes and their properties
Database dependency	Often database-specific syntax	Database independent
Return type	ResultSet (raw rows)	Java objects (e.g., List)
Case sensitivity	Table/column names usually case-insensitive	Class/property names are case-sensitive (Java rules)

Conclusion

HQL brings the simplicity of SQL-like querying into the object-oriented world of Hibernate, letting developers query, update, and delete data using Java class names and properties — while Hibernate handles the database-specific SQL translation automatically.